Lecture Notes for

Neural Networks I

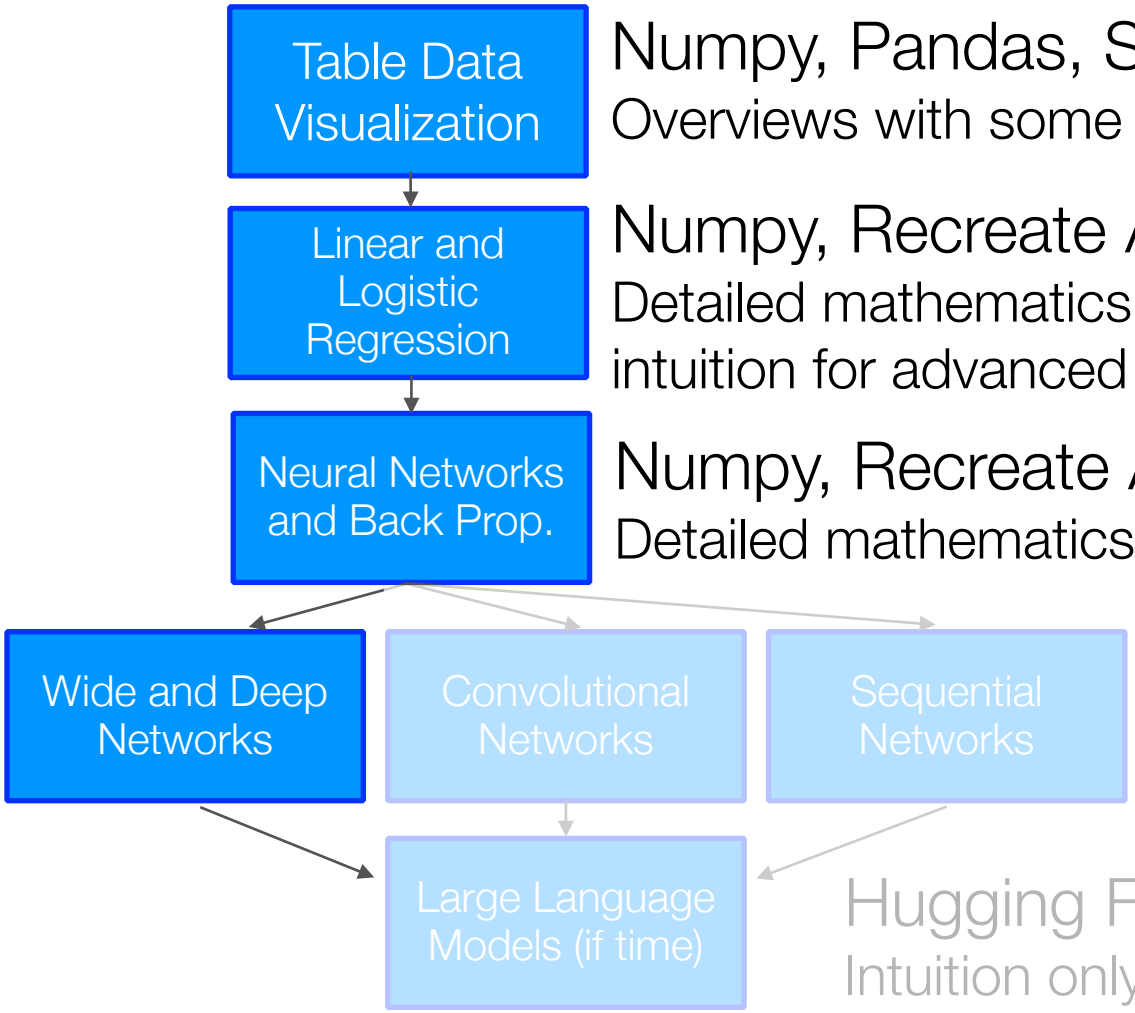


A History of Deep Learning And Introduction to Toolkits

Logistics and Agenda

- Logistics
  - Course schedule (including Final)
  - Grading update
- Agenda
  - Mid-term
  - Parameter Searching
  - Statistical Testing
  - Deep Learning History
  - Deep Learning toolkits

Class Overview, by topic



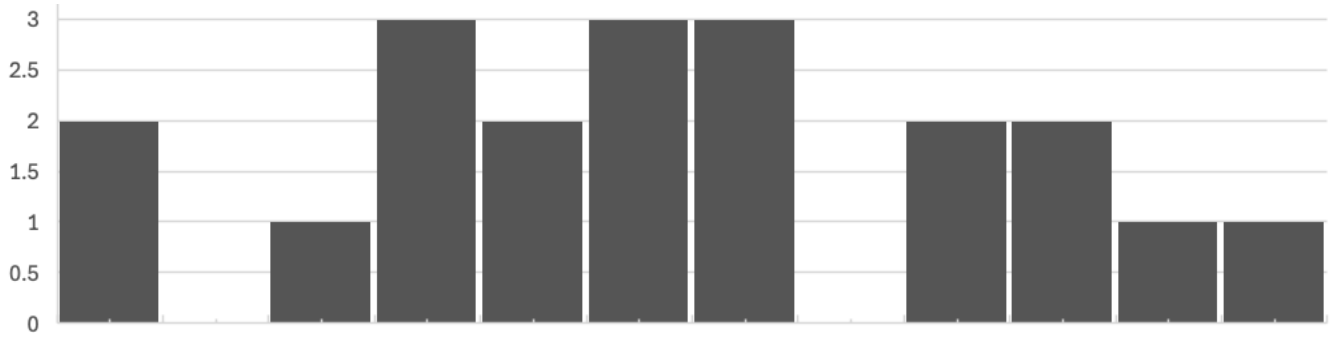
Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

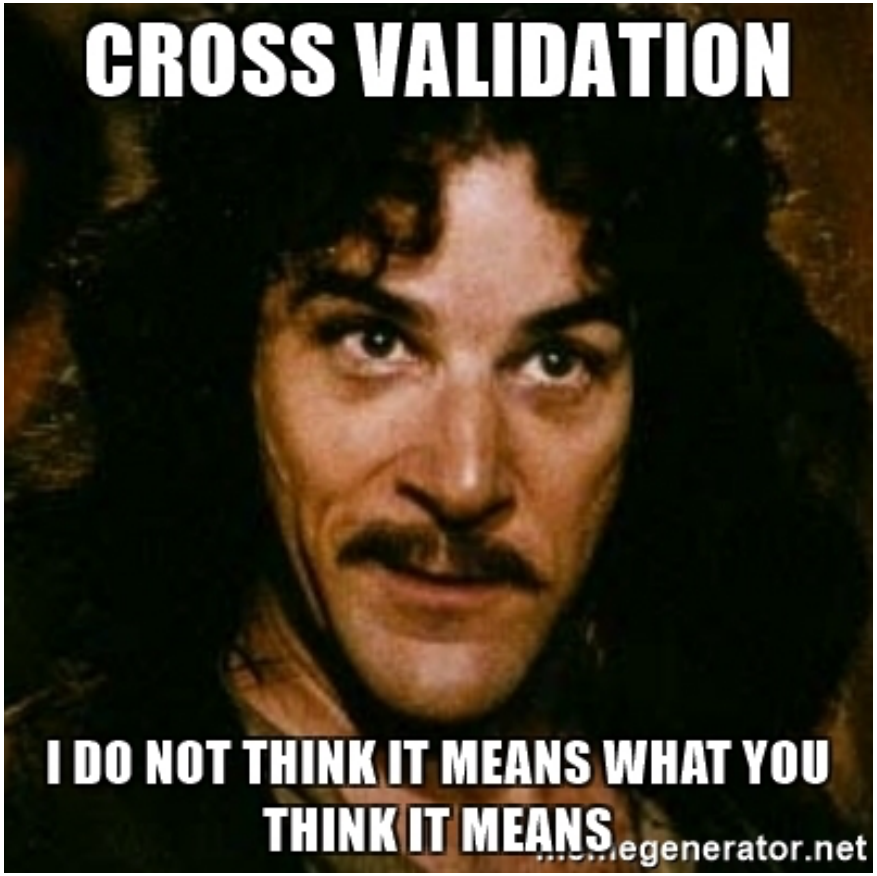
Hugging Face
Intuition only

Midterm Exam



No Curve:	50%	72%	96%
Yes Curve:	60%	78%	100%
	Low	Avg	High

Grid Searching Review



Review: Grid Searching

Trying to find the best parameters

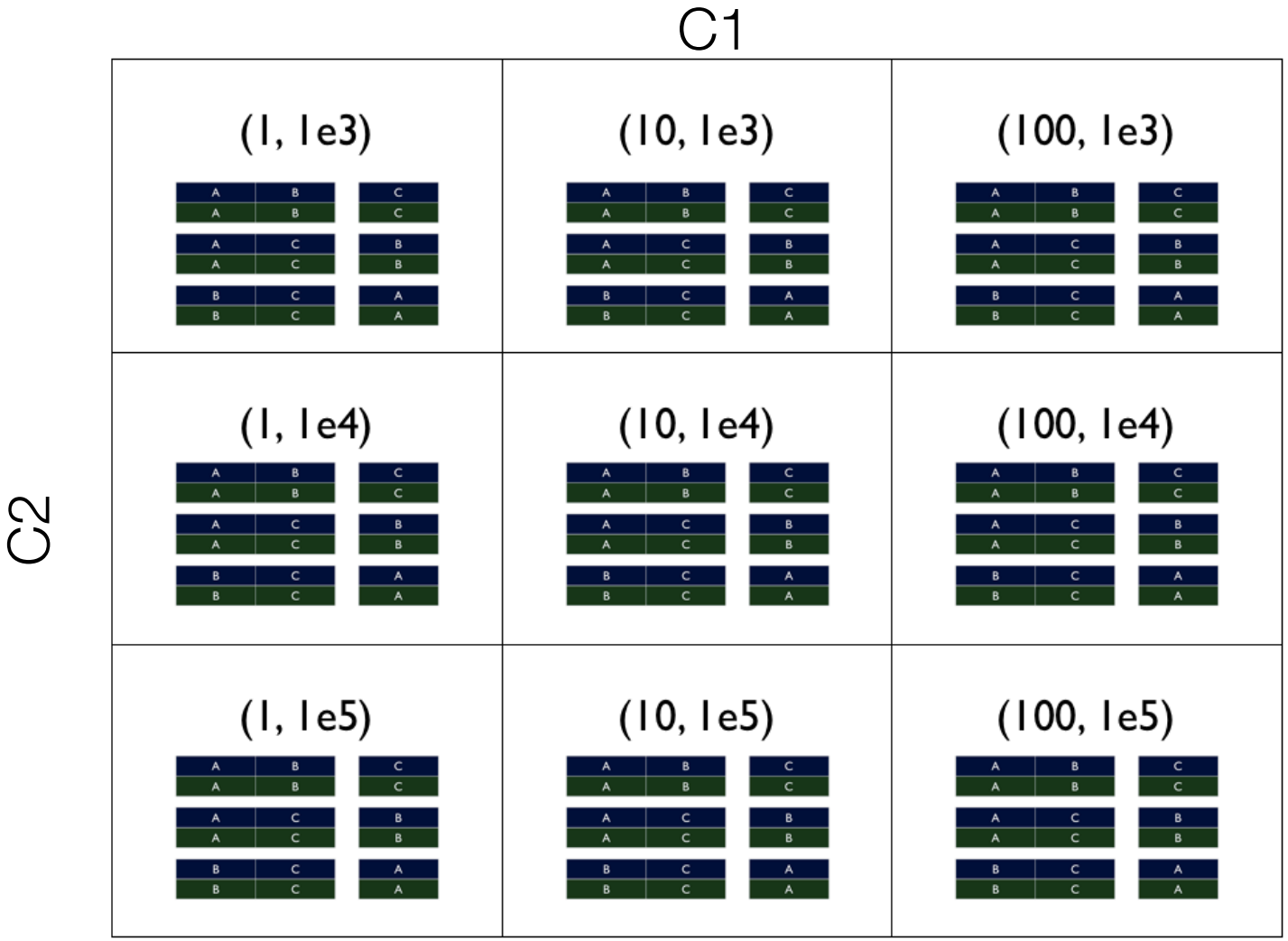
```
NN: C1=[1, 10, 100] C2=[1e3,1e4,1e5]
```

C1

C2			
	(1, 1e3)	(10, 1e3)	(100, 1e3)
	(1, 1e4)	(10, 1e4)	(100, 1e4)
	(1, 1e5)	(10, 1e5)	(100, 1e5)

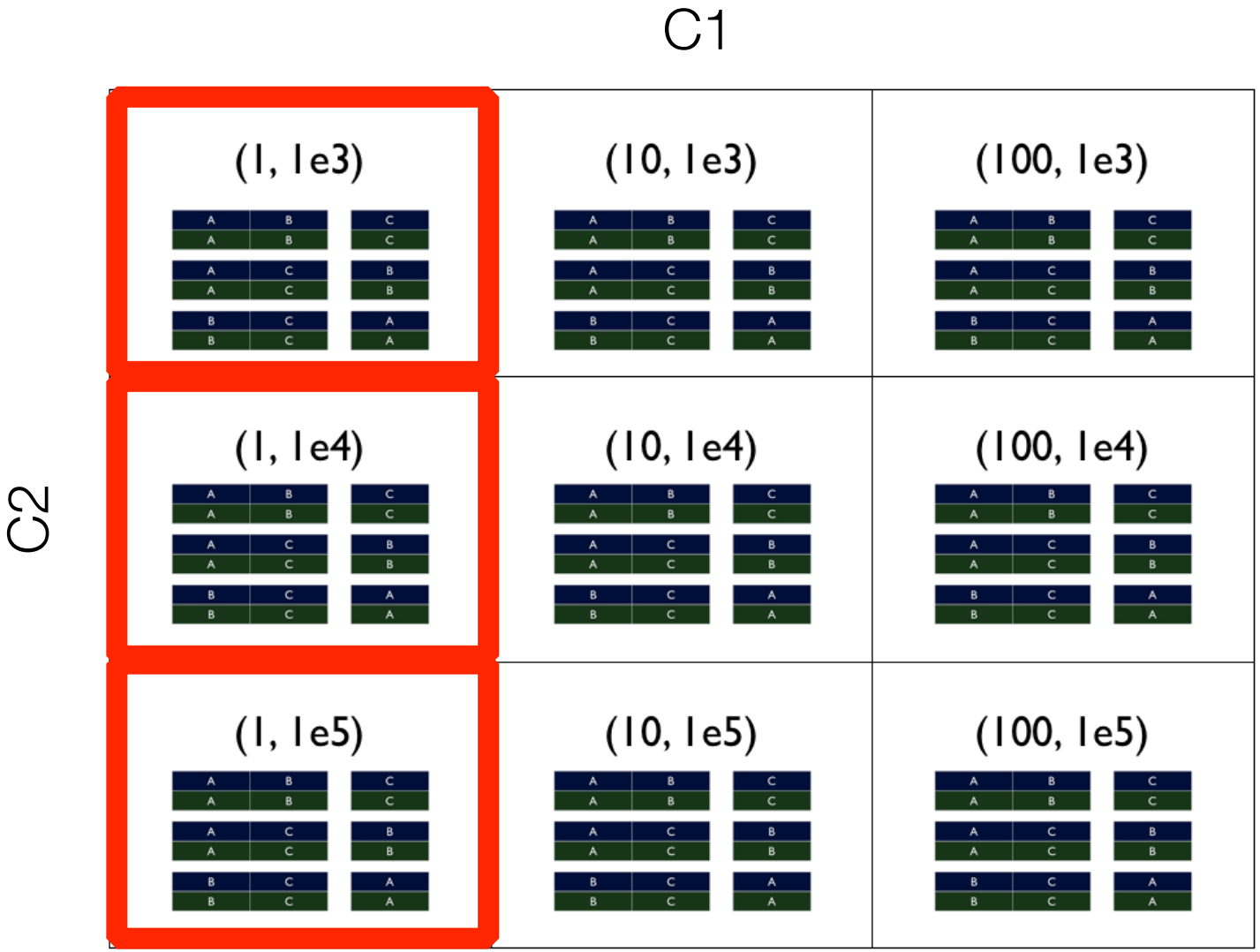
Review: Grid Searching

For each value, want to run cross validation…



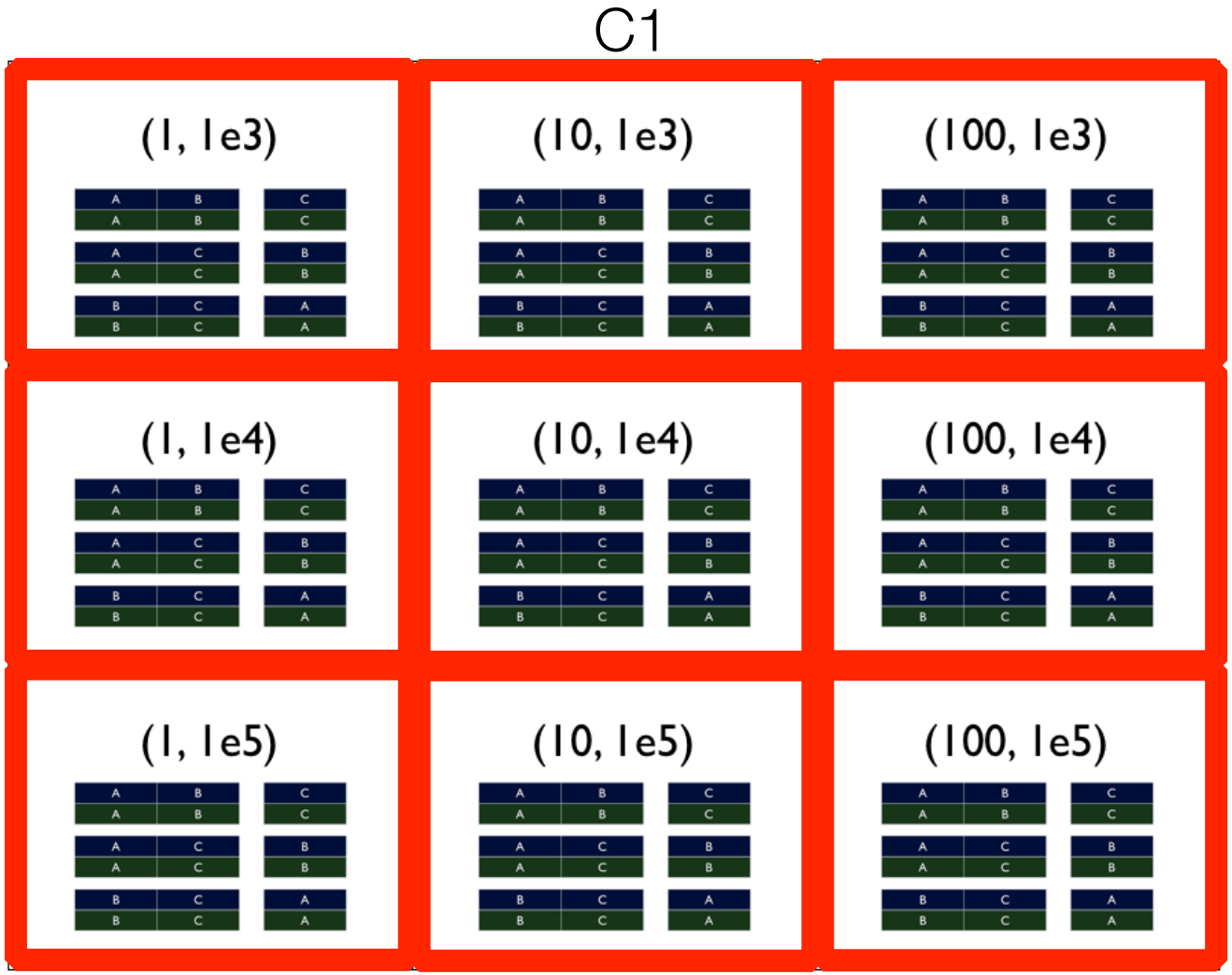
Review: Grid Searching

Could perform iteratively

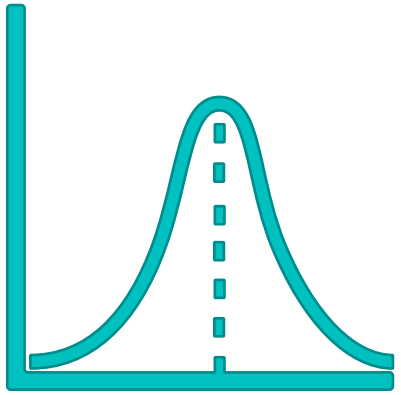


Review: Grid Searching

or at random…



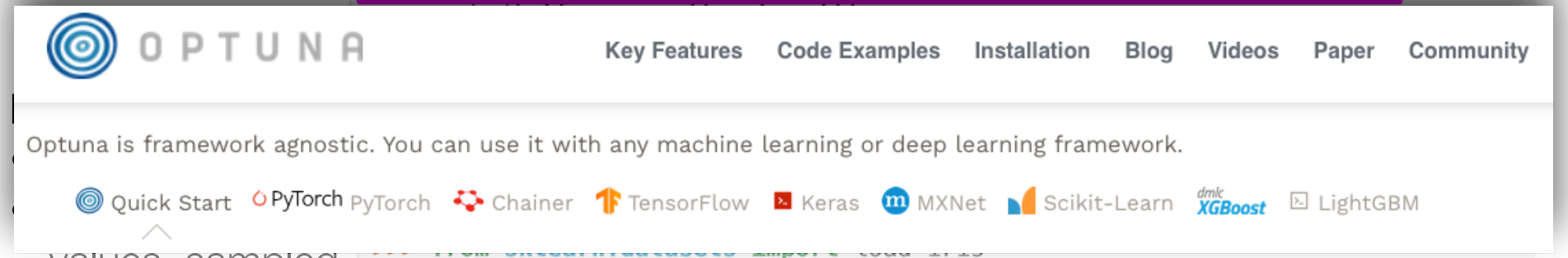
If random can also draw at random from a distribution!



C1 drawn from $\mathcal{N}(\mu = 50, \sigma = 20)$

Review: Grid Searches in Scikit-learn

```
>>> from sklearn import svm, datasets
>>> from sklearn.model_selection import GridSearchCV
>>> iris = datasets.load_iris()
>>> parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
>>> svc = svm.SVC()
>>> clf = GridSearchCV(svc, parameters)
>>> clf.fit(iris.data, iris.target)
GridSearchCV(estimator=SVC(),
             param_grid={'C': [1, 10], 'kernel': ('linear', 'rbf')})
```



values, sampled

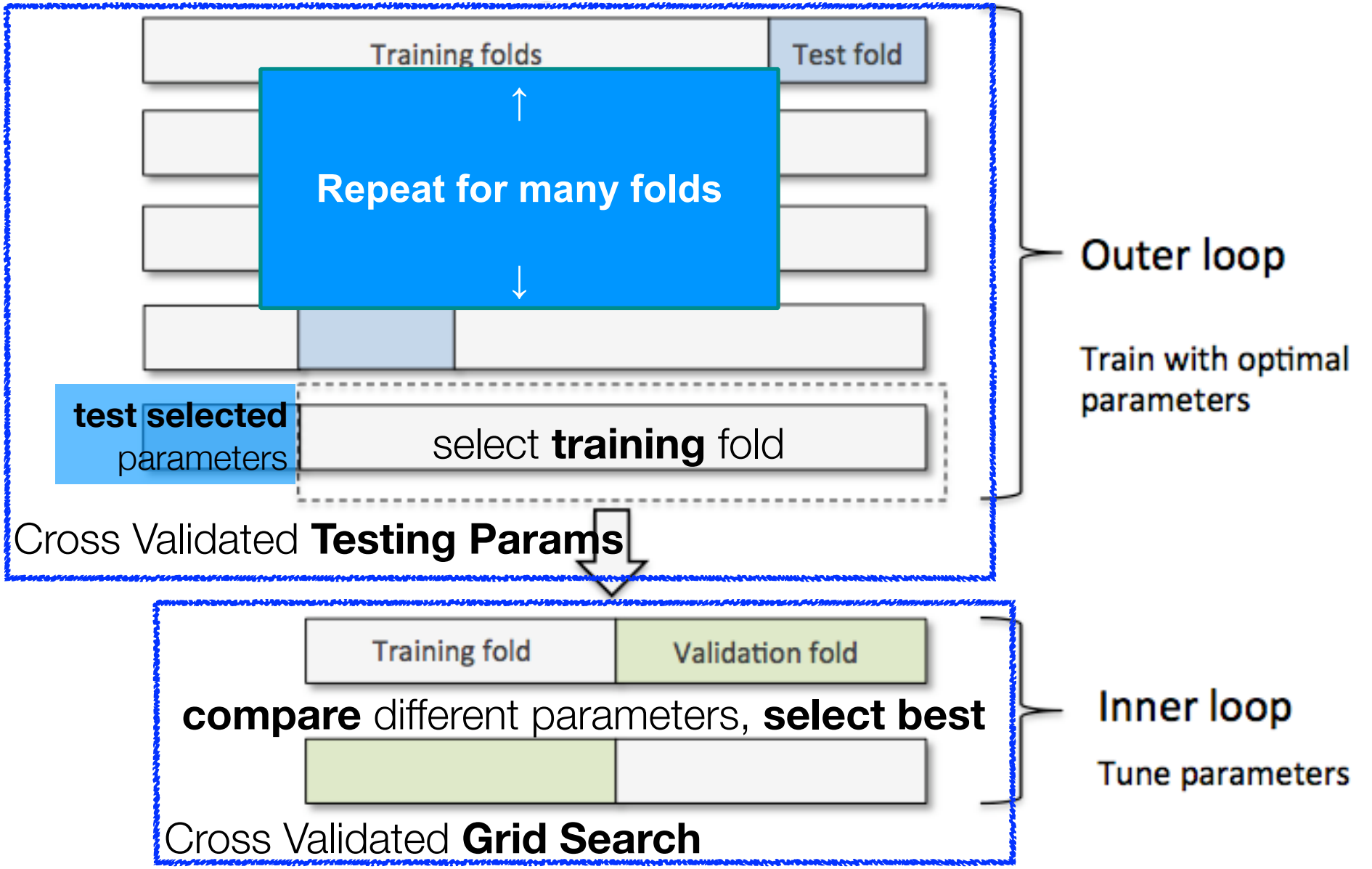
```
>>> from sklearn.linear_model import LogisticRegression
>>> from sklearn.model_selection import RandomizedSearchCV
>>> from scipy.stats import uniform
>>> iris = load_iris()
>>> logistic = LogisticRegression(solver='saga', tol=1e-2, max_iter=200,
...                               random_state=0)
>>> distributions = dict(C=uniform(loc=0, scale=4),
...                      penalty=['l2', 'l1'])
>>> clf = RandomizedSearchCV(logistic, distributions, random_state=0)
>>> search = clf.fit(iris.data, iris.target)
>>> search.best_params_
{'C': 2..., 'penalty': 'l1'}
```

Review: Self Test

- Using the grid search parameters and testing on the same set…
- Is this **data snooping**?
  - **A. True**, this is snooping because it uses test set to define parameters
  - **B. True**, this is snooping because we can no longer reliably define the expected performance on new data

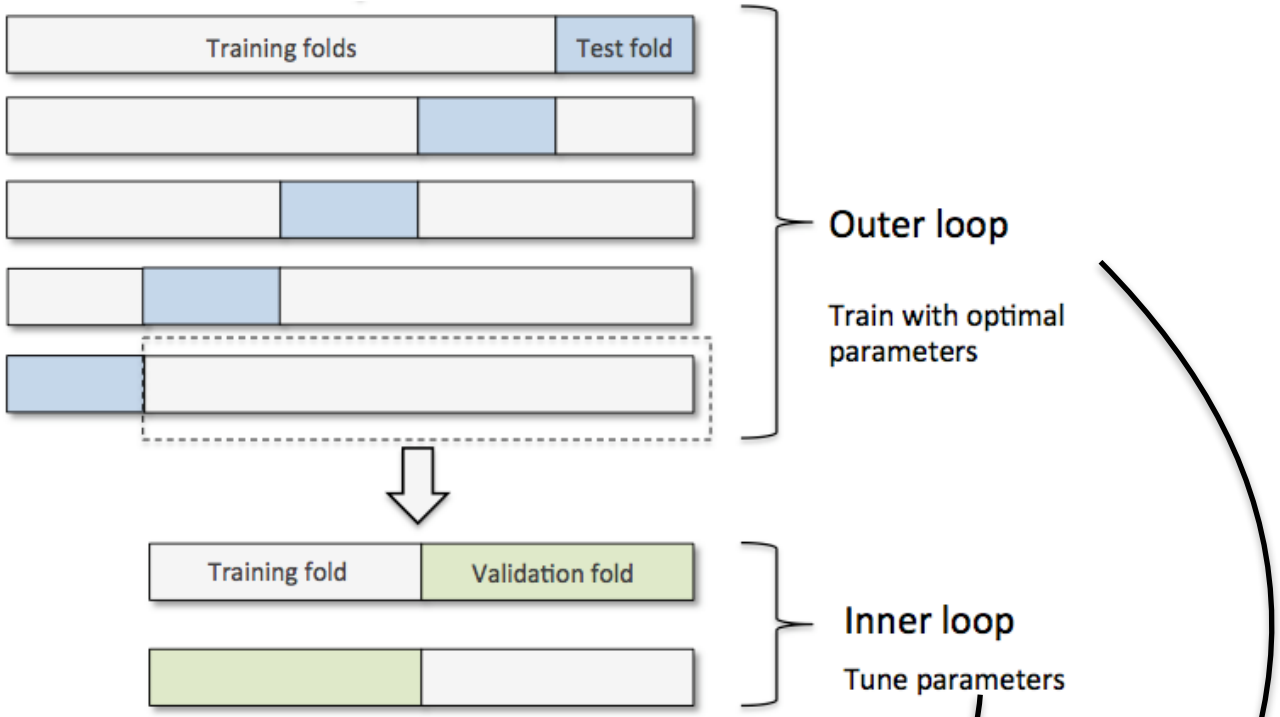
How can we define expected performance when using cross validation in a grid search?

Review: Nested Cross Validation



https://github.com/rasbt/python-machine-learning-book/

Review: Nested Cross Validation: Hyper-parameters




```
gs = GridSearchCV(estimator=pipe_svc,
                  param_grid=param_grid,
                  scoring='accuracy',
                  cv=2)
```

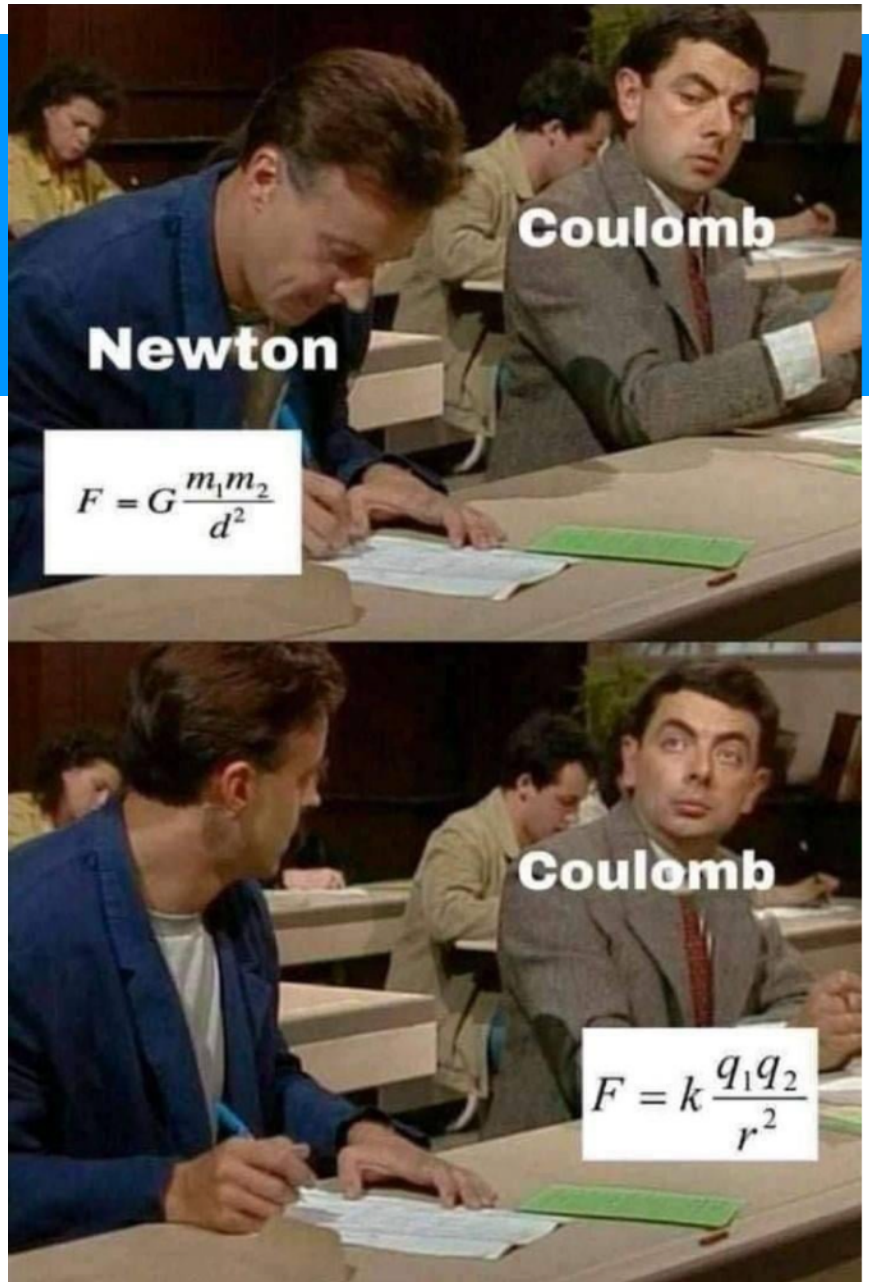
```
scores = cross_val_score(gs, X_train, y_train, scoring='accuracy', cv=5)
print('CV accuracy: %.3f +/- %.3f' % (np.mean(scores), np.std(scores)))
```

https://github.com/rasbt/python-machine-learning-book/

Self Test

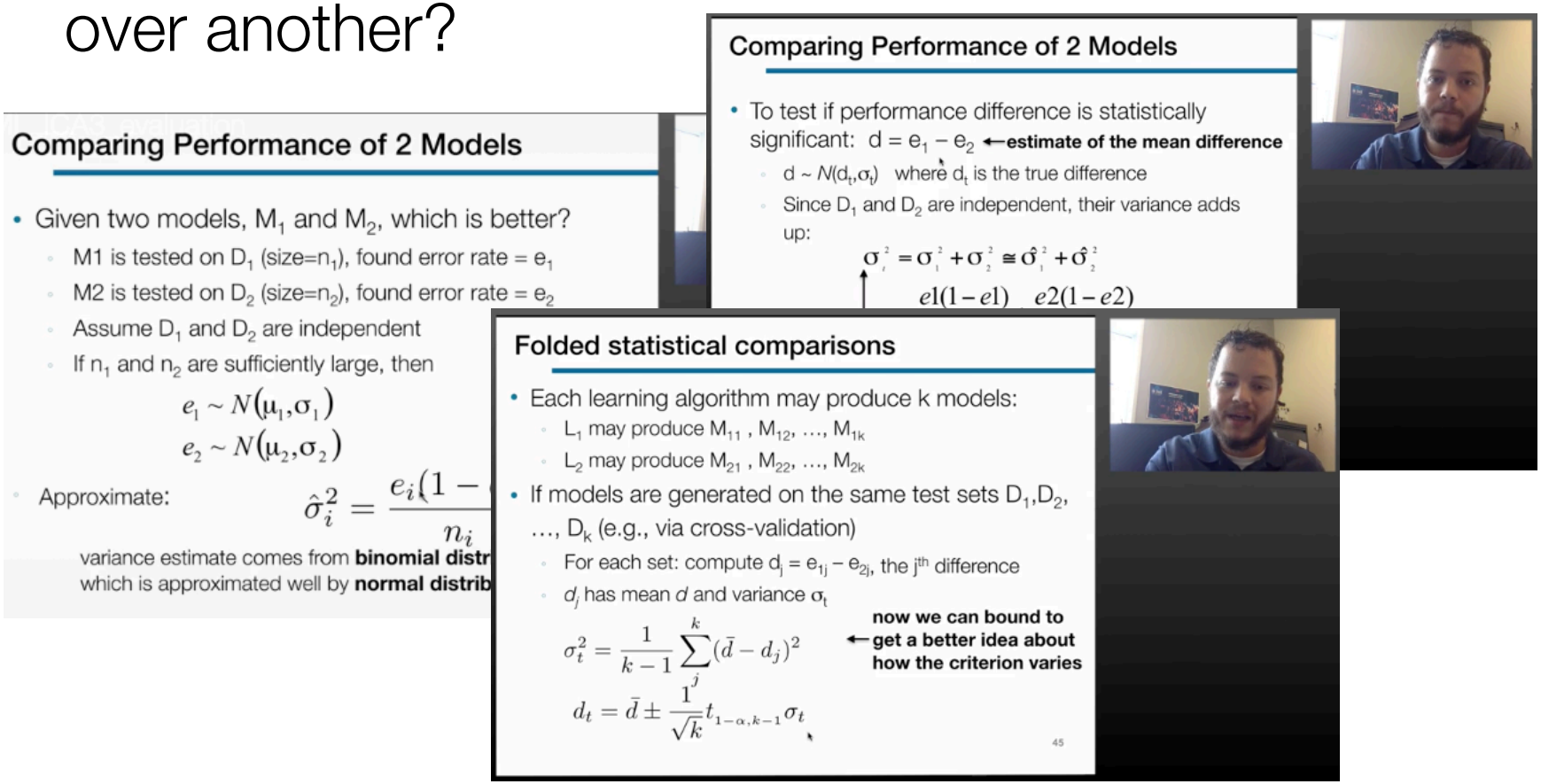
- **What is the end goal of nested cross-validation?**
  - A. To determine hyper parameters
  - B. To estimate generalization performance
  - C. To estimate generalization performance when performing hyper parameter tuning
  - D. To estimate the variation in tuned hyper parameters

Statistical Tests



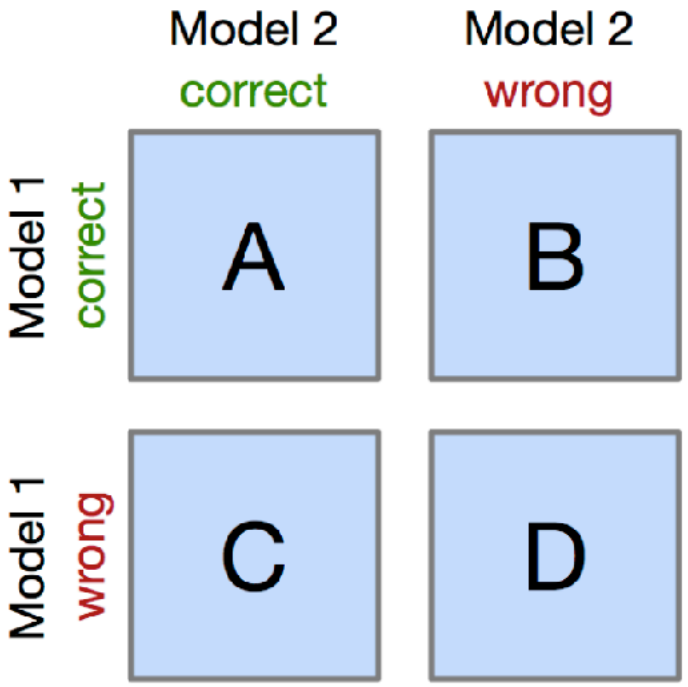
Flipped Module, Model Comparison

- How do we compare two (or more) trained models to one another?
- Are models different enough to prefer one model over another?



McNemar Testing for Comparing Performance

Few assumptions, **Null hypothesis**: predictions are not different!



McNemar and Edwards, 1948

$$\chi^2 \approx \frac{(|B - C| - 1)^2}{B + C}$$

If predictions are drawn from the same distributions, then this equation follows χ **squared statistic with one DOF**

Steps:

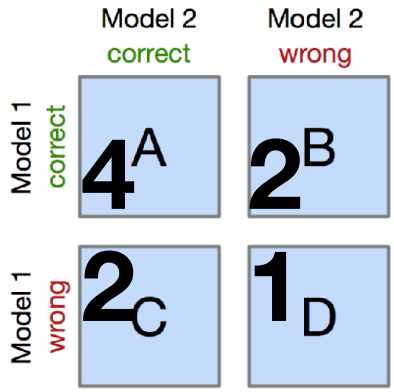
1. Compare each model's predictions on **the same test data** (2x2 matrix)
2. Calculate χ^2 statistic
3. Look up *critical value* associated with χ^2 statistic for given confidence
4. Are you confident enough to **reject the null hypothesis** that the performance is the same (p<0.05)?

One caveat: Statistical power depends upon B+C, which might be small, even with lots of test data.

https://sebastianraschka.com/blog/2018/model-evaluation-selection-part4.html

McNemar Example

Model 1	Model 2	Label	Matrix
T-shirt	T-shirt	T-shirt	A
Sneaker	T-shirt	Sneaker	B
T-shirt	Pullover	Pullover	C
Sneaker	Sneaker	Sneaker	A
T-shirt	Sneaker	Sneaker	C
Pullover	Pullover	T-shirt	D
Pullover	T-shirt	Pullover	B
Sneaker	Sneaker	Sneaker	A
Sneaker	Sneaker	Sneaker	A



McNemar and Edwards, 1948

$$\chi^2 \approx \frac{(|B - C| - 1)^2}{B + C}$$

$$\chi^2 = \frac{(|2 - 2| - 1)^2}{2 + 2} = 0.25$$

Confidence	0.90	0.95	0.99
1 DOF, Critical Value	2.706	3.841	6.635

https://www.itl.nist.gov/div898/handbook/eda/section3/eda3674.htm

Since 0.25 < 3.841, we cannot reject the null hypothesis. This means **we should not say the models' performance are different** based on the evidence.

Self Test:

- You have trained **three different models** on prediction of child poverty ratings. Each model is trained on the same data and **tested on the same single split** of the data.

- **Can you use a McNemar test for selecting a model?**
  - **A. Yes.** McNemar testing can be used for any testing of any model without any assumptions.
  - **B. Yes.** McNemar testing can be applied pairwise for the three models, comparing with three tests.
  - **C. No.** McNemar testing cannot be used for more then two models.
  - **D. No.** McNemar testing can only tell if the models are different, we still cannot tell which one is best

Some History of Deep Learning



Machine Learning Timeline (Neural Nets)

- Up to this point: back propagation ended first AI winter
- 80's, 90's: neural networks for image processing start to get deeper
  - but back propagation no longer efficient for training
  - Back propagation gradient **stagnates** research—can't train **deeper** networks

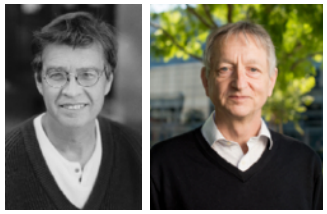
- Second AI winter begins, research in NN plummets
- Funding for and accepted papers with Neural Networks asymptotically approaches zero



1949, Hebb's Law
Close neuron fire together



1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



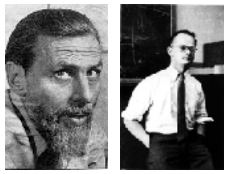
2003, Vapnik
Kernel SVMs



1940 | 1960 | 1980 | 2000 | 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

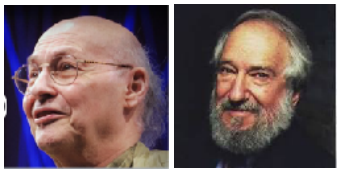
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests

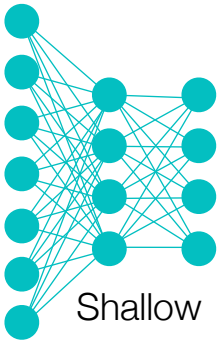


Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

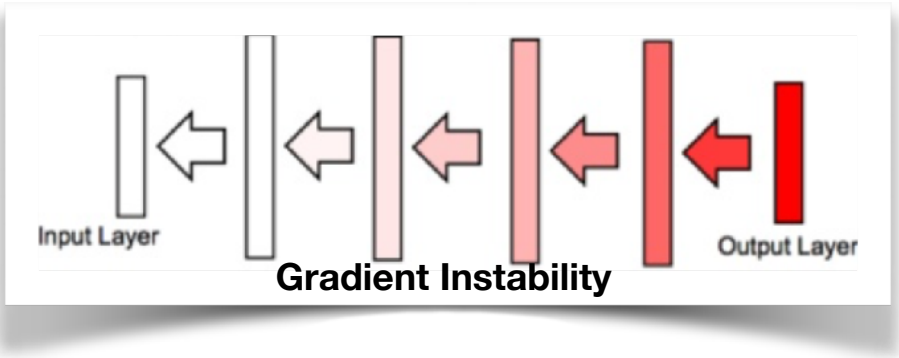
History of Deep Learning: Winter

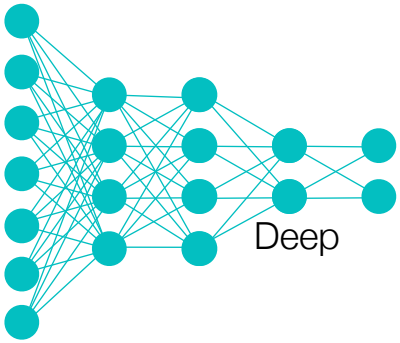


- AI Winter is coming:



Easy to train, performs on par with other methods





Hard to train, performs worse than other methods
~chance (untrainable)

1990's: Many companies founded on the promise of advanced ML and the internet…
2000's: Researcher have difficulty reconciling expressiveness with performance, cash in the market drops out, **no theory and no money**

Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

Machine Learning Timeline (Neural Nets)

- 2004: Hinton secures funding from CIFAR based on his reputation
  - *eventually*: Canada would be savior for neural networks
  - Hinton rebrands: **Deep Learning**
- 2006: Hinton publishes paper on using pre-training and Restricted Boltzmann Machines
- 2007: Another paper: Deep networks are more efficient when pre-trained
  - RBMs not really the important part



1949, Hebb's Law
Close neuron fire together



1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation

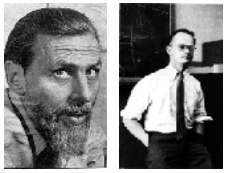


2003, Vapnik
Kernel SVMs

1940 | 1960 | 1980 | 2000 | 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

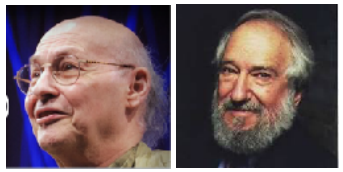
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



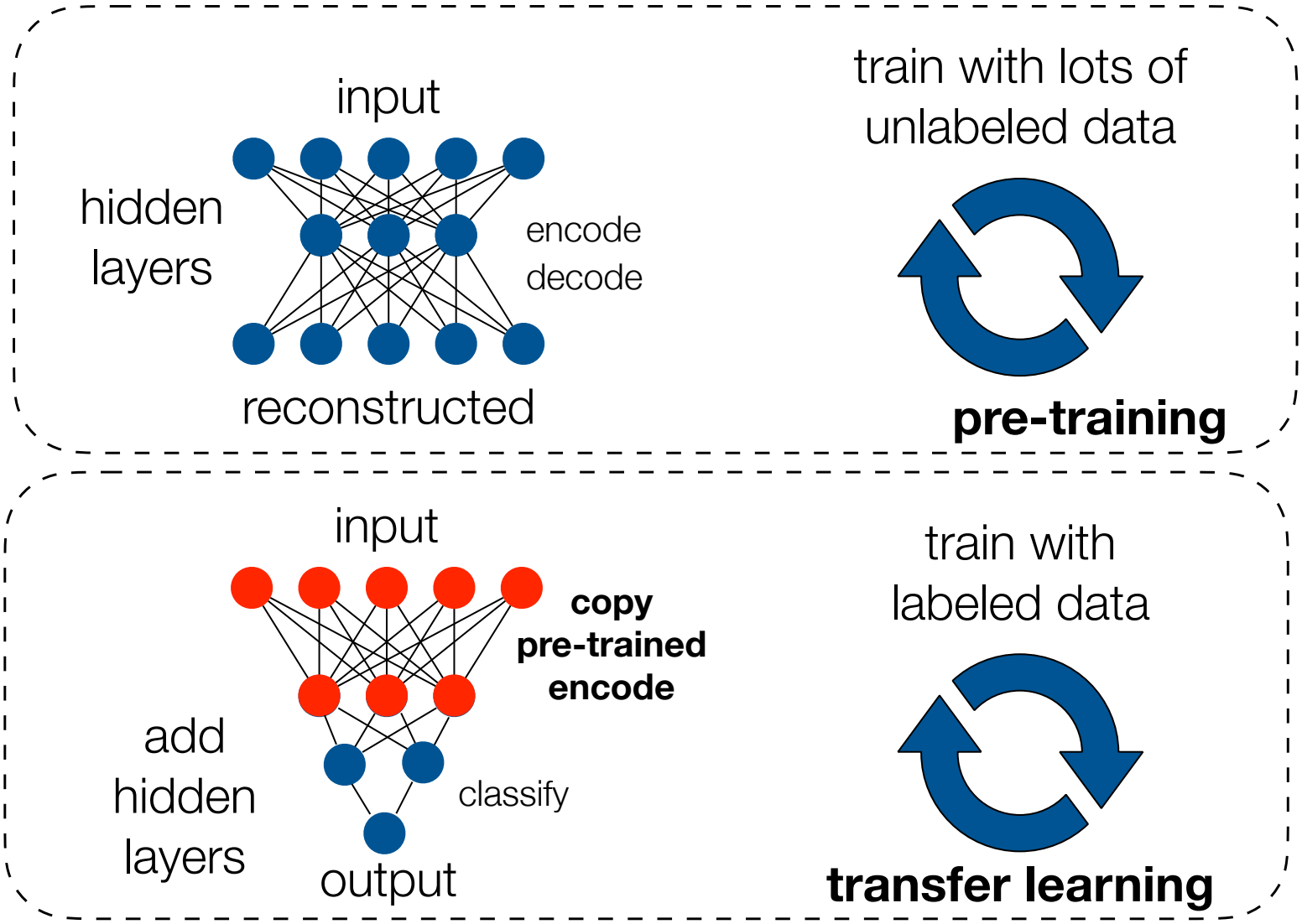
2001, Breiman
Random Forests



Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

Pre-training: still in the long winter

- auto-encoding (a form of pre-training)



Still in the Long Winter

- 2009: Hinton's lab starts using GPUs, Also Andrew Ng
  - GPUs decrease training time by 70 fold…
- 2010: Hinton's and Ng's students go to internships with Microsoft, Google, IBM, and Facebook



Navdeep Jaitly
Google Brain Team


George Dahl
Google Brain Team



Abdel-rahman Mohamed
Microsoft Research
Redmond, Washington | Computer Software

Current	Microsoft
Previous	University of Toronto, IBM, Microsoft
Education	University of Toronto


1949, Hebb's Law
Close neuron fire together



1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation

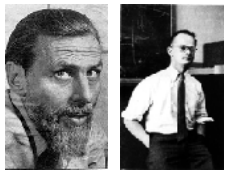

2003, Vapnik
Kernel SVMs

- Xbox Voice
- Android Speech Recognition
- IBM Watson
- DeepFace
- All of Baidu

1940 — 1960 — 1980 — 2000

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter

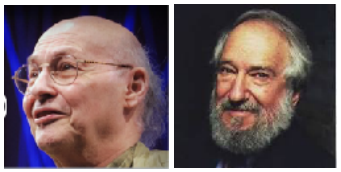
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



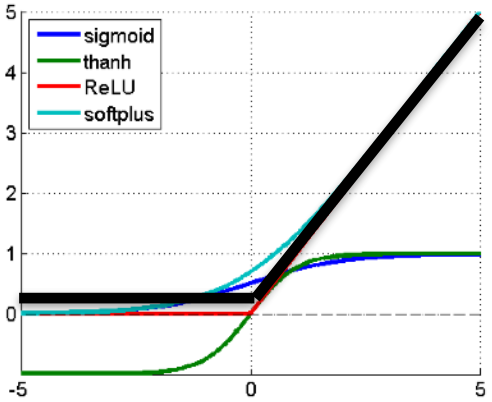
2001, Breiman
Random Forests



Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

Getting out of the long Winter (Review)

- 2011: Glorot and Bengio investigate more systematic methods for why past deep architectures did not work
  - **discover some interesting, simple fixes**: the type of neurons chosen and the selection of initial weights
  - do not require pre-training to get deep networks properly trained, just sparser representations and less complicated derivatives



ReLU: $f(x) = \max(0,x)$
$f'(x) = 1$ if $x>0$ else 0



1949, Hebb's Law
Close neuron fire together




1960, Widrow & Hoff
Adaline Network




1986, Rumelhart & Hinton
Back-propagation

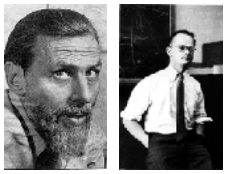


2003, Vapnik
Kernel SVMs

1940 — 1960 — 1980 — 2000 — 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

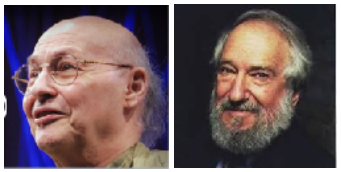
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

Machine Learning Timeline (

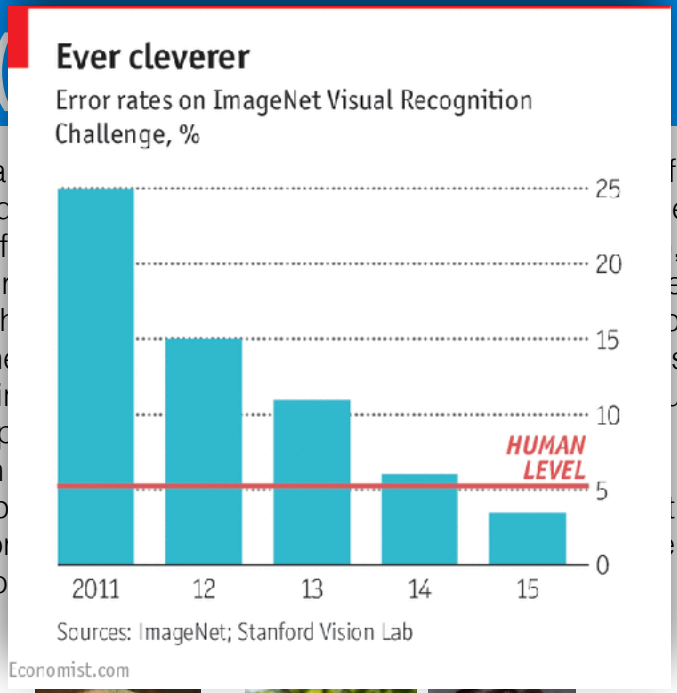
- **ImageNet competition occurs**
- **Second place**: 26.2% error rate
- **First place**:
  - From Hinton's lab, uses convolutional network with ReLU and dropout
  - 15.2% error rate
- Computer vision adopts deep learning with convolutional neural networks en mass



"I ha
hanc
last f
so I
as sh
come
pacir
happ
from
skep
a cor
Visio

first
e
,
ef
on
s
ust

t as
er

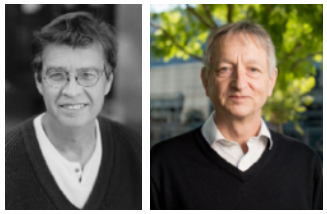




1949, Hebb's Law
Close neuron fire together



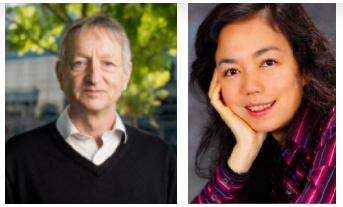
1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



2003, Vapnik
Kernel SVMs

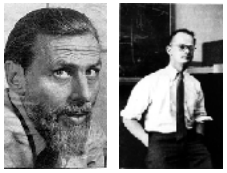


2012, Hinton, Fei-Fei Li
CNNs win ImageNet

1940 — 1960 — 1980 — 2000 — 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

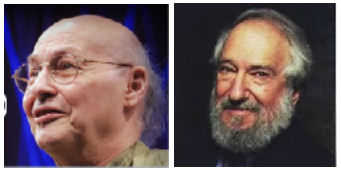
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

Machine Learning Timeline (Acoustics)

- 2012: Hinton Lab, Google, IBM, and Microsoft jointly publish paper, popularity for deep learning methods increases

Deep Neural Networks for Acoustic Modeling in Speech Recognition

[The shared views of four research groups]

[Geoffrey Hinton, Li Deng, Dong Yu, George E. Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N. Sainath, and Brian Kingsbury]

https://www.cs.toronto.edu/~gdahl/papers/deepSpeechReviewSPM2012.pdf



1949, Hebb's Law
Close neuron fire together



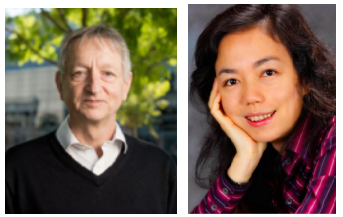
1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



2003, Vapnik
Kernel SVMs

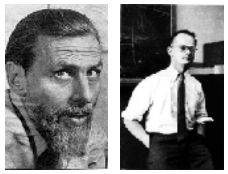


2012, Hinton, Fei-Fei Li
CNNs win ImageNet

1940 | 1960 | 1980 | 2000 | 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

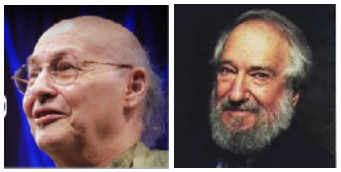
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

Machine Learning Timeline (Neural Nets)

- 2013: Andrew Ng and Google (BrainTeam)
  - run unsupervised feature creation on YouTube videos (becomes computer vision benchmark)



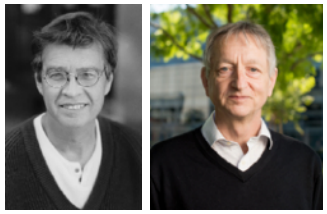
The work resulted in unsupervised neural net learning of an unprecedented scale - 16,000 CPU cores powering the learning of a whopping 1 billion weights. The neural net was trained on Youtube videos, entirely without labels, and learned to recognize the most common objects in those videos.



1949, Hebb's Law
Close neuron fire together



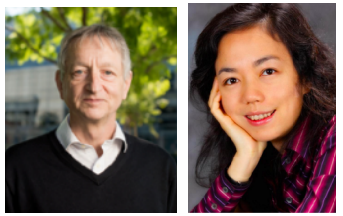
1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



2003, Vapnik
Kernel SVMs

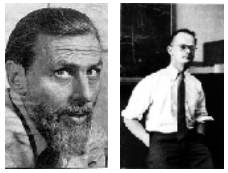


2012, Hinton, Fei-Fei Li
CNNs win ImageNet

1940 | 1960 | 1980 | 2000 | 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

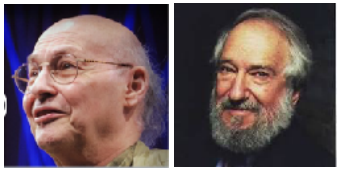
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



Read this: http://www.andreykurenkov.com/writing/a-brief-history-of-neural-nets-and-deep-learning/

A summary of the Deep Learning people:



Stayed at Univ. Montreal
Advises IBM



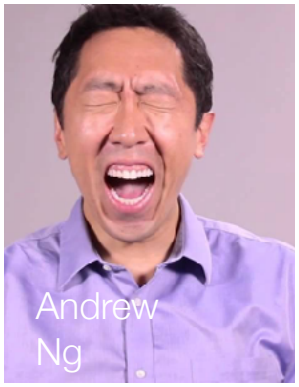
Heads Facebook AI Team



Univ. Toronto
Google



Stanford (HAI)
Former Chief Scien., AI/MLGoogle Cloud



Coursera
Baidu
Google



Stanford
Founded Coursera
MacArthur Genius



- Hinton: Restricted Boltzmann Machine, Deep autoencoder
- Bengio: neural language modeling.
- LeCun: Convolutional Neural Network
- NIPS, ICML, CVPR, ACL
- Google Brain, Deep Mind.
- FaceBook AI.

Made Deep Learning Instruction Accessible

deep learning

doi:10.1038/nature14539

ning

Geoffrey Hinton[4,5]

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and

Review of Deep Learning History

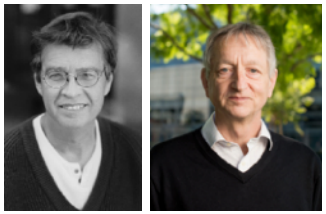
- Up to this point: back propagation saved AI winter for NN (Hinton and others!)
- 80's, 90's, 2000's: convolutional networks for image processing start to get deeper
  - but back propagation no longer does great job at training them
- SVMs and Random Forests gain traction…
  - The second AI winter begins, research in NN plummets
- 2004: Hinton secures funding from CIFAR in 2004 Hinton rebrands: Deep Learning
- 2006: Auto-encoding and Restricted Boltzmann Machines
- 2007: Deep networks are more efficient when pre-trained
- 2009: GPUs decrease training time by 70 fold…
- 2010: Hinton's students go to internships with Microsoft, Google, and IBM, making their speech recognition systems faster, more accurate and deployed in only 3 months…
- 2012: Hinton Lab, Google, IBM, and Microsoft jointly publish paper, popularity sky-rockets for deep learning methods
- 2011-2013: Ng and Google run unsupervised feature creation on YouTube videos (becomes computer vision benchmark)
- 2012+: Pre-training is not actually needed, just solutions for vanishing gradients (like ReLU, SiLU, initializations, more data, GPUs)



1949, Hebb's Law
Close neuron fire together



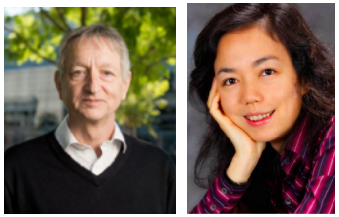
1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



2003, Vapnik
Kernel SVMs

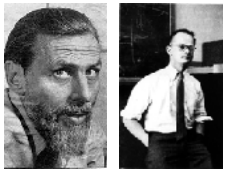


2012, Hinton, Fei-Fei Li
CNNs win ImageNet

1940 | 1960 | 1980 | 2000 | 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

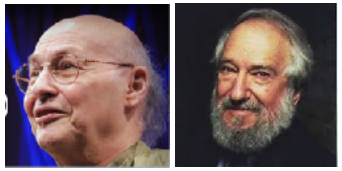
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



2015, Google
Tensorflow Open Source

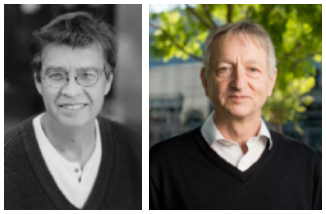
The New Paradigm



1949, Hebb's Law
Close neuron fire together



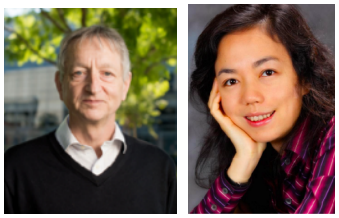
1960, Widrow & Hoff
Adaline Network



1986, Rumelhart & Hinton
Back-propagation



2003, Vapnik
Kernel SVMs



2012, Hinton, Fei-Fei Li
CNNs win ImageNet

1940 — 1960 — 1980 — 2000 — 2020

Period of Discovery | First AI Winter | Golden Age of NN | 2nd AI Winter | Age of Deep Learning

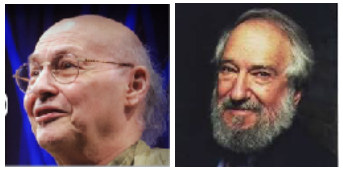
1943, McCulloch & Pitts
Logic Gates of The Mind



1957, Rosenblatt
Perceptron



1969, Minsky & Papert
Linear Models are Doomed



2001, Breiman
Random Forests



2011, Bengio
Init and ReLU



2015, Google
Tensorflow Open Source

TensorFlow

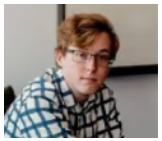




2015, Bengio, Schmidhuber
More Recurrent Nets

PyTorch
2017, PyTorch Beta

2018, Radford
GPT-1



Hugging Face
2020, Models Hub

2018 - 2022
Various GPT Papers

Lots of Progress to Discuss
Various Papers

3rd AI Winter?

future

2018 — 2020 — 2022 — 2024 — 2026

Age of Deep Learning | Age of Large Language Models



2017, Vaswani
Attention is All You Need

2020, Brown
Few Shot Language Models



ChatGPT

2022, OpenAI
ChatGPT

Hyped up promise of capability… and UBI

Self Test

- Modern day machine learning scientists have a rigorous background in neural biology or in identifying processes that are biologically plausible.
  - **A. True.** Neural networks are windows into the mind.
  - **B. True.** PhD's in Machine Learning require advanced biology study.
  - **C. True.** They have made bold claims about human intelligence, and therefore must have this expertise.
  - **D. False.** They receive no formal training in this nor do they regularly consult with experts in this field.

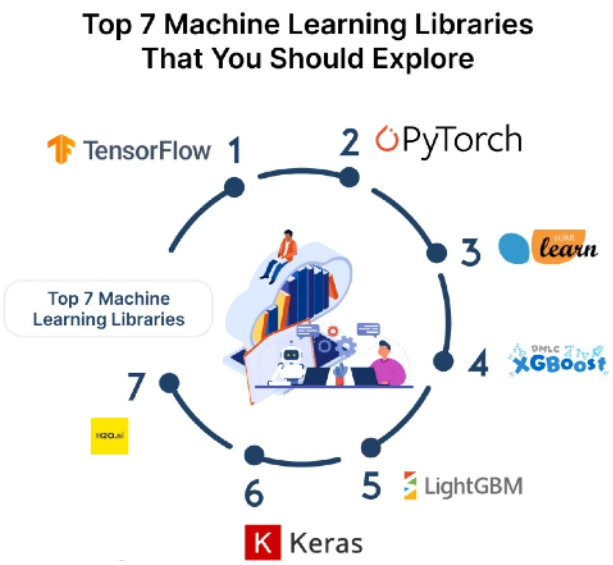
Tensor Packages for Deep Learning

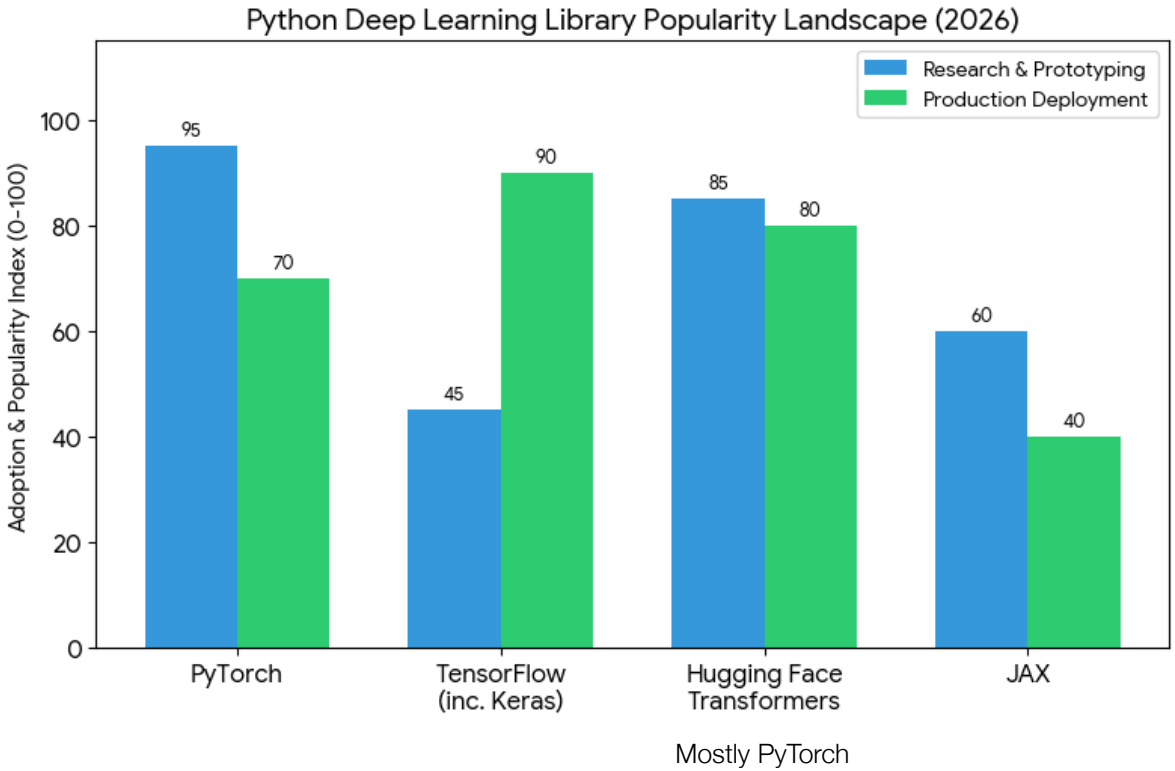
"Further discussion of it merely incumbers the literature and befogs the mind of fellow students."

- 2007: NIPS program committee rejects a paper on deep learning by *al. et.* Hinton because they already accepted a paper on deep learning and two papers on the same topic would be excessive.
- ~2009: A reviewer tells Yoshua Bengio that papers about neural nets have no place in ICML.
- ~2010: A CVPR reviewer rejects Yann LeCun's paper even though it beats the state-of-the-art. The reviewer says that it tells us nothing about computer vision because everything is learned.



Options for Deep Learning Toolkits





Tensorflow and PyTorch

- Open sourced library from Google
- Supports all major OS and Mobile OS
- Released November 9th, 2015
- Tensorflow 2.0 by Google Brain in 2019

- Open sourced library from FAIR (Facebook, mow Meta)
- Supports all major OS and Mobile OS
- Public Beta released 2017
- PyTorch 2.0 Released in 2023



In current 2.0 Releases, both have many similarities including eager execution and compiled execution



Programmatic creation

- Most toolkits use python to build a **computation graph** of operations
  - Build up computations
  - Execute computations
  - Similar to our implementation but entirely more optimized
- **Most Toolkits Support:**
  - tensor creation
  - functions on tensors
  - automatic differentiation

- Tensors are just multidimensional arrays
  - like in the Parameter class we made
  - but also optimized for GPU / CPU / TPU memory

```
class Parameter:
    def __init__(self, value):
        self.value = value
        self.grad = np.zeros_like(value)
```

```
class Loss:
    def __init__(self): pass
    def __call__(self, y_true, y_pred): pass
    def backward(self): pass
```

```
class Layer:
    def __init__(self):
        self.input = None
        self.output = None
    def backward(self, sensitivity_in): pass
    def parameters(self): return []
    def set_parameters(self, parameters): pass
    def zero_grad(self): pass
```

Tensor neural network functions

- Easy to define operations on Tensors

```
nn.Linear(in_features=20, out_features=5)
nn.Dropout(p=0.5)
nn.ReLU()
nn.LeakyReLU(negative_slope=0.1)
nn.Sigmoid()
nn.Softmax(dim=-1)
nn.Conv1d(in_channels=3, out_channels=16, kernel_size=3)
nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3)
nn.MaxPool2d(kernel_size=2, stride=2)
nn.CrossEntropyLoss()
…
```

- Each function uses **Automatic Differentiation**
- potentially has CPU and GPU implementations

Computation Graph with Code

```
import torch
X = torch.tensor( … )
y = torch.tensor( … )
```

$$J(\mathbf{W}) = \frac{1}{N}\sum_{i}^{N}(y^{(i)} - \underbrace{(\mathbf{W}\cdot\mathbf{x}^{(i)} + \mathbf{b})}_{y_{pred}})^2$$

1. **Setup** Variables and computations

```
W = torch.randn((1,num_features), requires_grad=True)
b = torch.randn((1, 1), requires_grad=True)

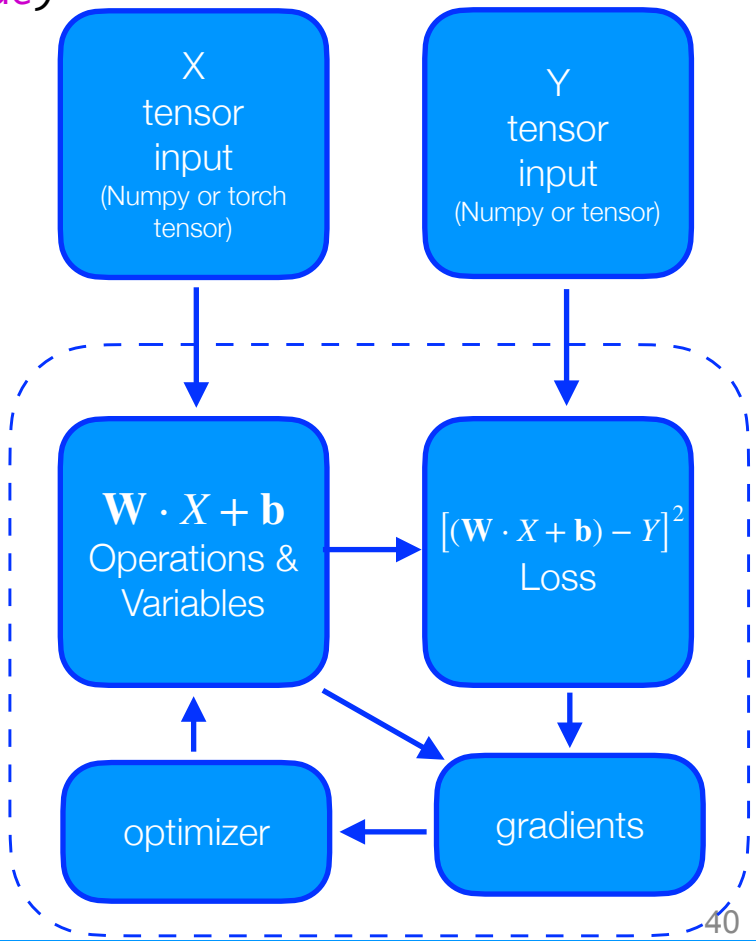
def feedforward(X,y):
    y_pred = torch.matmul(X,W) + b
    loss = torch.mean((y-y_pred)**2)
    return loss
```

2. Add **optimization** operation to computation graph
   Adjusts variables (W, b) to minimize loss with auto differentiation

```
opt = torch.optim.SGD([W, b], lr=0.01)
opt.zero_grad()

loss = feedforward(X_numpy, y_numpy)
loss.backward()
opt.step()
```

3. **Run graph operation** once, → one optimization update on all variables



http://www.datasciencecentral.com/profiles/blogs/google-open-source-tensorflow

End of Session

- Next Time:
  - Introduction to PyTorch, by fire
  - Embeddings
  - Wide and Deep Networks